DOCUMENTAÇÃO

DO PROJETO

Sistema de Gestão de Restaurantes

Tech Challenge - Fase 1

Arquitetura e Desenvolvimento Java

Abril 2026

Sumário

(Dica: Clique com botão direito no sumário e selecione "Atualizar Campo" para atualizar os numeros de pagina)

1. Processos de Desenvolvimento **3**

1.1 Metodologia 3

1.2 Fluxo de Trabalho (Git) 3

1.3 Definition of Done 3

2. Requisitos **4**

2.1 Requisitos Funcionais 4

2.2 Requisitos Não-Funcionais 4

2.3 Stack Tecnologica 5

3. Técnicas de Levantamento **6**

4. UML - Introdução **7**

5. Diagrama de Caso de Uso **8**

6. Documentação de Casos de Uso **9**

7. Divisão de Atividades **11**

8. Estrutura de Pacotes **14**

9. Configuração Docker **15**

10. Collection Postman **17**

11. Cronograma **18**

12. Checklist de Entrega **19**

1. Processos de Desenvolvimento

1.1 Metodologia

Modelo: Scrum adaptado para grupo de 5 pessoas

Sprints: 2 semanas cada

Daily: 15 minutos (Discord/Meet)

Review: Ao final de cada sprint

1.2 Fluxo de Trabalho (Git)

```
main (protegida)
 └── develop
     ├── feature/usuario-crud
     ├── feature/autenticacao
     ├── feature/docker-infra
     └── feature/tests-qa
```

1.3 Definition of Done (DoD)

☐ Código revisado (Pull Request aprovado por pelo menos 1 membro)

☐ Testes unitários passando (cobertura mínima 70%)

☐ Documentação Swagger atualizada

☐ Testes de integração no Postman validados

☐ Container Docker rodando sem erros

☐ Flyway migration aplicada com sucesso

2. Requisitos

2.1 Requisitos Funcionais (RF)

ID	Requisito	Prioridade
RF-001	Cadastrar, atualizar e excluir usuários	Alta
RF-002	Troca de senha em endpoint separado	Alta
RF-003	Atualização de dados em endpoint distinto	Alta
RF-004	Registrar data da última alteração	Alta
RF-005	Buscar usuários pelo nome	Média
RF-006	Garantir unicidade do e-mail	Alta
RF-007	Validar login (login e senha)	Alta
RF-008	Suportar dois tipos de usuário	Alta
RF-009	Documentação com Swagger/OpenAPI	Alta
RF-010	Versionamento de API	Média

2.2 Requisitos Não-Funcionais (RNF)

ID	Requisito	Tecnologia
RNF-001	Arquitetura por Domínio	Spring Boot
RNF-002	Banco de dados relacional	MySQL 8.0
RNF-003	Conteinerização	Docker + Compose
RNF-004	Versionamento de banco	Flyway
RNF-005	Mapeamento ORM	Spring Data JPA
RNF-006	Validação de dados	Bean Validation
RNF-007	Padronização de erros	RFC 7807
RNF-008	Testes automatizados	JUnit 5 + Mockito
RNF-009	Mapeamento DTO-Entity	MapStruct

2.3 Stack Tecnológica Confirmada

Camada	Tecnologia	Versão
Backend	Spring Boot	3.2.x
Web	Spring MVC	-
Segurança	Spring Security	6.x
Dados	Spring Data JPA	-
Banco	MySQL	8.0
Migrações	Flyway	10.x
Validação	Hibernate Validator	8.x
Testes	JUnit 5 + Mockito	-
DTOs	MapStruct	1.5.x
Documentação	SpringDoc OpenAPI	2.x

3. Técnicas de Levantamento de Requisitos

3.1 Técnicas Utilizadas

Técnica	Aplicação	Responsável
Análise de Documento	Tech Challenge fornecido	Todos
Brainstorming	Definição de regras de negócio	Todos
Entrevista Simulada	Necessidades dos restaurantes	PO
Prototipagem Rápida	Collection Postman	QA/Infra
Análise de Domínio	Entidades e relacionamentos	Tech Lead

3.2 Resultado das Técnicas

Entidades Identificadas

Usuario: classe base abstrata ou superclasse

- Cliente (especialização)
- Dono Restaurante (especialização)

Atributos Obrigatórios

nome: String
email: String (unico)
login: String
senha: String
dataUltimaAlteracao: LocalDateTime
endereco: Endereco (objeto embeddable ou entidade)

Estrutura do Endereço

Atributo	Tipo	Obrigatoriedade
rua	String (VARCHAR)	Obrigatorio
numero	String (VARCHAR)	Obrigatorio
cidade	String (VARCHAR)	Obrigatorio
estado	String (VARCHAR 2)	Obrigatorio

Atributo	Tipo	Obrigatoriedade
cep	String (VARCHAR 9)	Obrigatório
complemento	String (VARCHAR)	Opcional

4. UML - Introdução

4.1 Propósito

A UML (Unified Modeling Language) será utilizada para: visualizar a estrutura do sistema antes da implementação; comunicar a arquitetura entre os membros do grupo; documentar decisões de design; e facilitar a manutenção futura.

4.2 Diagramas a Serem Produzidos

Diagrama	Objetivo	Responsável
Caso de Uso	Funcionalidades do sistema	Analista
Classes	Estrutura das entidades	Arquiteto
Sequência	Fluxo de autenticação	DEV Backend
Componentes	Arquitetura Docker	QA/Infra
Implantação	Deploy com Composê	QA/Infra

5. Diagrama de Caso de Uso

5.1 Atores Identificados

Ator	Descrição
Cliente	Usuário que consulta restaurantes, faz pedidos
Dono de Restaurante	Usuário que gerencia seu estabelecimento
Sistema	Componente automatico (validações, envios)

5.2 Diagrama de Caso de Uso (Estrutura)

Ator: Cliente

UC-001: Cadastrar-se
UC-002: Fazer Login
UC-003: Buscar Usuários
UC-004: Atualizar Dados (exceto senha)
UC-005: Alterar Senha
UC-006: Excluir Conta

Ator: Dono de Restaurante

UC-001 a UC-006 (mesmos do Cliente)

UC-007: Gerenciar Estabelecimento (futuro)

Ator: Sistema (Automático)

UC-008: Validar unicidade de e-mail
UC-009: Registrar data de alteração
UC-010: Versionar API
UC-011: Padronizar respostas de erro (RFC 7807)

6. Documentação de Casos de Uso

6.1 UC-001: Cadastrar Usuário

Campo	Descrição
Nome	Cadastrar Usuário
Ator Principal	Cliente ou Dono de Restaurante
Descrição	Permite que novos usuários se cadastrem
Pré-condições	Sistema disponível; Email não existente
Pós-condições	Usuário criado; Data registrada

Fluxo Principal (Happy Path)

1. Usuario acessa endpoint POST /api/v1/usuarios
2. Sistema recebe JSON com: nome, email, login, senha, endereco, tipo
3. Sistema valida campos obrigatorios (@NotBlank, @Email, @Size)
4. Sistema verifica unicidade do e-mail
5. Sistema criptografa senha (BCrypt)
6. Sistema persiste usuário no banco (dataUltimaAlteracao = now())
7. Sistema retorna 201 Created com Location header

6.2 UC-002: Fazer Login

Campo	Descrição
Nome	Fazer Login
Ator Principal	Cliente ou Dono de Restaurante
Descrição	Validar credenciais do usuário
Pré-condições	Usuário deve estar cadastrado

6.3 UC-004: Atualizar Dados do Usuário

Campo	Descrição
Nome	Atualizar Perfil
Endpoint	PUT /api/v1/usuarios/{id}

Campo	Descrição
Restrição	Não permite alterar senha

6.4 UC-005: Alterar Senha

Campo	Descrição
Nome	Alterar Senha
Endpoint	PATCH /api/v1/usuarios/{id}/senha
Restrição	Endpoint exclusivo para senha

7. Divisão de Atividades - Equipe de 5 Pessoas

Membro 1: QA e Infraestrutura com Docker

Responsabilidade	Descrição	Entregáveis
Infraestrutura Docker	Estrutura de containers	Dockerfile, compose
Banco de Dados	MySQL, Flyway	Migrations
Quality Assurance	Collection Postman	JSON exportado
Testes de Integração	Validar endpoints	Relatório

Membro 2: Tech Lead / Arquiteto

Responsabilidade	Descrição
Estrutura do Projeto	Arquitetura por domínio
Configuração Spring Boot	application.yml, profiles
Entidades JPA	Mapeamento das entidades
Flyway Strategy	Estratégia de migrations
Code Review	Revisar todos os PRs

Membro 3: Backend DEV - CRUD e Validações

Responsabilidade	Descrição
CRUD Usuário	Controllers, Services, Repositories
DTOs	Request/Response com MapStruct
Validações	Bean Validation customizadas
Tratamento de Erros	@ControllerAdvice + ProblemDetail

Membro 4: Backend - Autenticação e Segurança

Responsabilidade	Descrição
Login/Auth	Serviço de validação de credenciais

Responsabilidade	Descrição
Spring Security	Configuração do Security FilterChain
JWT	Implementação do desafio extra
Criptografia	BCryptPasswordEncoder

Membro 5: Documentação e Testes Unitários

Responsabilidade	Descrição
Swagger/OpenAPI	Documentação completa da API
Testes Unitários	JUnit 5 + Mockito
README	Documentação do projeto
Relatório Técnico	PDF final para entrega

8. Estrutura de Pacotes (Arquitetura por Dominio)

```
src/main/java/com/postech/techchallenge/
├── domain/
│   ├── usuario/
│   │   ├── controller/UsuarioController.java
│   │   ├── service/UsuarioService.java
│   │   ├── repository/UsuarioRepository.java
│   │   ├── model/entity/Usuario.java (abstract)
│   │   ├── dto/request/UsuarioRequestDTO.java
│   │   └── mapper/UsuarioMapper.java
│   └── auth/
├── config/
├── exception/
└── infrastructure/
```

9. Configuração do Docker

9.1 Dockerfile

```
# Build stage
FROM maven:3.9-eclipse-temurin-17-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Runtime stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

9.2 Flyway Migration

Campo	Tipo
id	BIGINT AUTO_INCREMENT PRIMARY KEY
nome	VARCHAR(100) NOT NULL
email	VARCHAR(100) NOT NULL UNIQUE
login	VARCHAR(50) NOT NULL
senha	VARCHAR(255) NOT NULL
tipo	VARCHAR(20) NOT NULL
data_ultima_alteracao	TIMESTAMP NOT NULL

10. Collection Postman (Estrutura Sugerida)

```
📂 Tech Challenge - Fase 1
├── 📂 1 - Usuarios
│   ├── POST Cadastrar Cliente
│   ├── POST Cadastrar Dono Restaurante
│   ├── GET Buscar por Nome
│   ├── PUT Atualizar Dados
│   ├── PATCH Alterar Senha
│   └── DELETE Excluir Usuario
├── 📂 2 - Autenticacao
├── 📂 3 - Cenarios de Erro
└── 📂 4 - Health & Docs
```

11. Cronograma Sugerido (3 Sprints de 2 Semanas)

Sprint 1: Fundacao (Semanas 1-2)

Responsavel	Entrega
QA/Infra	Docker Compose funcionando, banco rodando
Tech Lead	Projeto base, entidades JPA mapeadas
DEV CRUD	Estrutura de controllers, DTOs definidos
DEV Auth	Configuração inicial de seguranca
DEV Docs	Swagger configurado

Sprint Goal: Aplicação sobe no Docker, endpoints básicos respondem

Sprint 2: Funcionalidades (Semanas 3-4)

Responsavel	Entrega
QA/Infra	Collection Postman completa
Tech Lead	Flyway migrations, ajustes
DEV CRUD	CRUD completo com validações
DEV Auth	Login funcionando (com ou sem JWT)
DEV Docs	Testes unitários iniciados

Sprint Goal: Todos os requisitos funcionais implementados

Sprint 3: Qualidade e Entrega (Semanas 5-6)

Responsavel	Entrega
QA/Infra	Testes de integração, validação final
Tech Lead	Code review final, otimizações
DEV CRUD	Tratamento de erros (Problem Detail)
DEV Auth	Segurança finalizada
DEV Docs	Relatório técnico PDF, README completo

Sprint Goal: Sistema pronto para entrega, documentação completa

12. Checklist de Entrega

Codigo (GitHub/GitLab)

☐ Código fonte completo
☐ README.md com instrucoes
☐ docker-compose.yml funcional
☐ Collection Postman (/docs/postman/)
☐ Swagger UI acessível em /swagger-ui.html

Relatório Técnico (PDF)

☐ Descrição da arquitetura
☐ Modelagem de entidades (diagrama)
☐ Endpoints com exemplos
☐ Prints do Swagger
☐ Prints da collection Postman
☐ Estrutura do banco (tabelas)
☐ Passo a passo Docker Compose

Documento gerado em: Abril 2026

Versão: 1.0

Projeto: Tech Challenge - Fase 1 | Arquitetura Java

Tech Challenge - Fase 1

Arquitetura e Desenvolvimento Java

Grupo de 5 integrantes